AI-Based Smart Navigation System

Prakhar Anand

Department of Artificial Intelligence and Machine Learning
Vellore Institute of Technology, Bhopal

Course: Artificial Intelligence & Machine Learning

Instructor: Bandla Pavan Babu

Date: 25 March 2026

Abstract

This project is designed as a Smart Navigation System using AI techniques that determine the shortest path between two points using the A* search algorithm. It is designed to calculate the time taken using a mathematical estimation technique. It is a simulation of the actual navigation systems used in the real world and is a demonstration of the Artificial Intelligence and Machine Learning techniques used in it.

Introduction

Artificial Intelligence is a technology used by machines to perform tasks intelligently. Navigation tools like GPS and Google Maps are examples of AI applications. They are used to calculate the best path between two points. The objective of this project is to create a smart navigation system that can efficiently find the shortest path between two points.

REAL-WORLD MOTIVATION

Reason for this project is to try solving the problems encountered in one's everyday movements from place to place. The systems that help people navigate are basically based around algorithms created by computer programmers. A popular example of this type of system is Google Maps. The purpose of this project is to provide a basic understanding of how such systems use artificial intelligence principles to solve these types of navigation problems.

Problem Statement

The problem of finding the most efficient route between two cities is a common problem in real-life situations. The conventional method might not be able to address the optimization of the route effectively. An intelligent system has to be used to find the best route and estimate the travel time.

Objectives

To implement A* search algorithm

To create a map representation using a graph structure

To find the shortest path using a specific algorithm

To estimate travel time using a specific algorithm

To simulate intelligent navigation

Methodology

The following is the methodology used in this project:

Represent the map as a graph structure

Assign a cost to each edge of the map

Implement A* algorithm

Use heuristic values for optimization purposes

Calculate the total distance using a specific algorithm

Predict travel time using a specific algorithm

Algorithms Used

*A *Algorithm*

A* is an informed search algorithm that finds the shortest path using:

- g(n): Actual cost from start
- h(n): Heuristic estimate
- f(n) = g(n) + h(n)

BFS (Breadth-First Search)

Explores nodes level by level and guarantees shortest path in unweighted graphs.

DFS (Depth-First Search)

Explores nodes deeply before backtracking.

Implementation

- Programming Language: Python
- Data Structure: Graph (Dictionary)
- Library Used: heapq

The system takes input nodes, processes them using A* algorithm, and outputs the shortest path along with distance and estimated time.

Input:

Start Node: A

Goal Node: F

Output:

Shortest Path: A → B → D → F

Total Distance: 4 units

Estimated Time: 6 minutes

```python
import heapq

graph = {
    'A': [('B', 1), ('C', 3)],
    'B': [('A', 1), ('D', 1), ('E', 4)],
    'C': [('A', 3), ('E', 1)],
    'D': [('B', 1), ('F', 2)],
    'E': [('B', 4), ('C', 1), ('F', 1)],
    'F': [('D', 2), ('E', 1)]
}

heuristic = {
    'A': 5, 'B': 3, 'C': 4,
    'D': 2, 'E': 1, 'F': 0
}

def a_star(start, goal):
    pq = [(0, start)]
    g_cost = {node: float('inf') for node in graph}
    g_cost[start] = 0
    parent = {start: None}

    while pq:
        _, current = heapq.heappop(pq)

        if current == goal:
            break

        for neighbor, weight in graph[current]:
            new_cost = g_cost[current] + weight

            if new_cost < g_cost[neighbor]:
                g_cost[neighbor] = new_cost
                f_cost = new_cost + heuristic[neighbor]
                heapq.heappush(pq, (f_cost, neighbor))
                parent[neighbor] = current
```

```python
    path = []
    node = goal

    if node not in parent:
        return None, float('inf'), None

    while node is not None:
        path.append(node)
        node = parent[node]

    path.reverse()
    return path, g_cost[goal], parent


def predict_time(distance):
    speed = 40
    return round((distance / speed) * 60, 2)


# MAIN
print("\n🕒 Smart Navigation System")
print("Nodes:", list(graph.keys()))

start = input("Enter Start Node: ").upper()
goal = input("Enter Goal Node: ").upper()

if start not in graph or goal not in graph:
    print("❌ Invalid input!")
else:
    path, dist, parent = a_star(start, goal)
```

```python
if path:
    print("\n✅ SHORTEST PATH FOUND")
    print("➡ Path:", " → ".join(path))

    print("\n📊 Step-by-step:")
    total = 0
    for i in range(len(path)-1):
        current = path[i]
        next_node = path[i+1]

        # find distance between nodes
        for neighbor, weight in graph[current]:
            if neighbor == next_node:
                print(f"{current} → {next_node} = {weight}")
                total += weight

    print("\n📏 Total Distance:", total)
    print("⏱ Estimated Time:", predict_time(total), "minutes")

else:
    print("❌ No path found!")
```

Results :

Start Node: A
Goal Node: F
Shortest Path: A -> B -> D -> F
Step-by-Step Distance:
A -> B: 1 unit
B -> D: 1 unit
D -> F: 2 units
Total Distance: 4 units
Estimated Time: 6.0 minutes

```
🧭 Smart Navigation System
Nodes: ['A', 'B', 'C', 'D', 'E', 'F']
Enter Start Node: A
Enter Goal Node: F

✅ SHORTEST PATH FOUND
➡Path: A → B → D → F

📊 Step-by-step:
A → B = 1
B → D = 1
D → F = 2

📏 Total Distance: 4
⏱Estimated Time: 6.0 minutes
PS C:\Users\hp\Desktop\Coding>
```

WORKING FLOW

Our workflow includes:
Step 1: Enter your 'Start' and 'Goal' nodes into the database
Step 2: Initialize the graph with distance values
Step 3: Execute the A* algorithm
Step 4: Calculate the shortest distance between all nodes in the graph
Step 5: Display the distance travelled (actual) and the estimated time to travel the distance.

CO Mapping

CO	Description
CO1	Intelligent agent
CO2	Search algorithms (A*)
CO3	Distance & estimation
CO4	Time prediction
CO5	Real-world application

Advantages

Efficient path finding

Real world application

Easy to implement

Scalable

Limitations

No real-time traffic data

Static graph

Simple prediction model

Real-time traffic and weather data are not a part of this system, it uses an older or static graph.

Future Scope

Adding real-time traffic data

Using sophisticated ML models

Adding GUI

Adding GPS

It may be possible to develop enhancements that include real-time flows as well as an interface with visual representations of the data.

Conclusion

The Smart Navigation System showcases the application of artificial intelligence towards resolving real-world issues that involve route optimization. The A* Algorithm is a competent route resolution algorithm, capable of calculating the optimal distance and hence the time required for travel. This project served as a learning tool for general understanding of search algorithms and their uses. Additionally, this project emphasizes the role that artificial intelligence plays in today's navigation systems.

References

Artificial Intelligence: A Modern Approach

Python Documentation

Online AI tutorials